

Frontend Requirements Specification: Sri Lankan G.C.E. Advanced Level SaaS Tuition Platform

Executive Summary

The modern educational landscape in Sri Lanka, particularly the highly competitive G.C.E. Advanced Level (A/L) private tuition sector, requires a digital infrastructure capable of handling high-throughput student interactions, complex administrative hierarchies, and stringent digital rights management (DRM) to protect intellectual property. This document serves as the authoritative Frontend Requirements Specification for a comprehensive Software-as-a-Service (SaaS) platform tailored to this market. By translating the established Product Requirements Document into precise frontend execution guidelines, this specification provides a rigid blueprint for a development team utilizing React, TypeScript, Tailwind CSS, and React Native (via Expo). The architectural approach bifurcates state management, delegating asynchronous server state to React Query (TanStack Query) while handling synchronous UI and client state via Zustand. Furthermore, the specification details the implementation of sophisticated local contextual features, including QR-based hall attendance systems capable of processing hundreds of students in minutes, bank slip upload mechanisms to accommodate local payment behaviors, and dynamic Z-Score projection dashboards that contextualize student performance against national statistical distributions¹.

1. Information Architecture

The information architecture of the platform is designed to support a multi-tenant SaaS model where the complexity of the interface scales dynamically based on the authenticated user's role. The routing strategy relies on React Router for the web application, utilizing the `createBrowserRouter` API to support nested routing, data loaders, and error boundaries, while the mobile application leverages React Navigation with a hybrid drawer and bottom-tab structure.

The route structure strictly segregates public interfaces from protected educational and administrative environments. Public routes encompass the tenant-specific landing pages, the global authentication flows, and registration portals. The authentication pathways are particularly complex, featuring a unified `/login` route that branches into an OTP verification flow, a device limit warning interceptor, and a password reset sequence. Protected routes, nested under an `/app` namespace, are guarded by a high-order component that verifies the presence and validity of a JWT stored in memory, cross-referencing it with the user's role claims to authorize access to specific sub-routes. The dashboard navigation acts as the central hub, utilizing a dynamic sidebar on the web that reads from a centralized configuration object to render only the links permitted by the user's role. Nested routes are heavily utilized within the `/app/courses` and `/app/exams` paths, allowing the interface to maintain persistent navigation

elements while seamlessly swapping out contextual sub-views, such as moving from a course syllabus overview directly into the immersive video player or PDF viewer.

2. User Roles

The platform accommodates twelve distinct user roles, each requiring a bespoke frontend experience tailored to their operational scope. The frontend must parse the user's role from the authentication token and dynamically construct the navigation tree, inject the appropriate dashboard widgets, and hide unauthorized elements.

Role	Dashboard Focus	Permissions & Visibility	Hidden Menus	Quick Actions & Widgets
Super Admin	Global platform analytics, tenant health, overarching revenue metrics.	Full visibility across all tenants, ability to impersonate users, system-wide broadcasting.	None.	Suspend tenant, Broadcast system alert, View global infrastructure health widget.
Institute Owner	Institute-wide revenue, total active student count, branch performance comparisons.	Full visibility within their specific tenant, access to financial reports, staff management.	Super Admin global settings, Platform billing.	Add new branch, Export financial master report, Revenue projection widget.
Teacher	Upcoming class schedules, aggregate course analytics, assignment submission queues.	Course creation, grading, syllabus management, direct student communication.	Institute financials, Global staff management, Cashier interfaces.	Create announcement, Start live Zoom class, Student engagement heatmap widget.
Assistant Teacher	Doubt resolution queues, grading	Access to specific assigned courses, ability	Course creation, Financials, Staffing,	Grade pending paper, Reply to student doubt, Grading

	interfaces for essay assignments.	to grade and comment on student submissions.	Institute analytics.	progress widget.
Branch Manager	Real-time branch attendance, daily fee collection totals, local staff monitoring.	Branch-specific attendance overrides, manual payment approvals, visitor logs.	Cross-branch data, Global financials, Course creation.	Override attendance, Approve pending bank slip, Daily collection widget.
Cashier	Live fee collection interface, invoice generation, bank slip verification queue.	Point of Sale (POS) interface, search by student ID, payment history viewing.	Courses, Exams, Educational content, Staff management.	Accept manual payment, Print thermal receipt, Pending slip verification widget.
Receptionist	Visitor registration, rapid QR scanning interface, student PVC ID card printing.	Access to basic student profiles, class schedules, and attendance logging.	All financials, Content creation, Examination management.	Scan student QR, Register walk-in inquiry, Hall capacity widget.
Finance Officer	Revenue reconciliation, payment gateway settlement logs, refund processing.	Advanced financial reporting, manual transaction overrides, fee structure adjustments.	Educational content, Exams, Attendance tracking, Branch management.	Export CSV ledger, Process refund, Gateway health widget.
Content	Video and PDF	Access to	Financials,	Upload video

Creator	upload queues, DRM configuration, metadata management.	media library, tagging systems, and content scheduling.	Attendance, Live class management.	batch, Add watermark overlay, Content encoding status widget.
Moderator	Discussion forum oversight, live chat moderation, flagged content review.	Ability to delete comments, issue temporary bans, review AI chat logs.	Financials, Exams, Course creation, System settings.	Ban user, Delete flagged comment, Active reports widget.
Student	Active course progress, upcoming exam schedule, Z-Score analytics.	Access to enrolled courses, payment portal, personal performance metrics.	All administrative and teaching menus.	Join live class, Download PDF notes, Continue learning widget, Z-Score tracking widget.
Parent	Dependent's attendance logs, upcoming fee deadlines, comparative exam performance.	Read-only access to dependent's data, communication preferences.	All administrative, teaching, and direct learning menus.	Pay outstanding fees, Download progress report, Attendance calendar widget.

3. Complete Screen Inventory

The frontend architecture necessitates a highly structured screen inventory to ensure the React and React Native teams account for every visual state. Every screen must be engineered to handle variations in network latency, data availability, and input errors without degrading the

user experience.

Screen Name	User Role	Layout Structure	Core Components	Interactive States (Empty, Error, Loading, Success)	Responsive Behaviour
Authentication (Login/Register)	All Roles	Split-screen (Hero graphic left, Form right on desktop)	React Hook Form, Zod validation logic, Shadcn Inputs, OTP Modal	Loading: Spinner in submit button. Error: Inline red text below fields. Success: Confetti animation and redirect.	Stacks vertically on mobile viewports; hero graphic is hidden on screens under 768px.
Admin Analytics Dashboard	Institute Owner	Asymmetric Grid Layout (12-column constraint)	Recharts (Line/Bar/Area), Metric StatCards, Recent Activity DataTable	Loading: Full-page customized Skeleton grid. Empty: Lottie animation prompting data generation.	1 column mobile, 2 columns tablet, 4 columns desktop.
Student Learning Hub	Student	Masonry Grid (Course Cards)	Progress Ring, Swipeable Course Cards, Z-Score Widget	Empty: "Enroll in a course" Call to Action. Loading: Card skeletons.	Cards become horizontally swipeable carousels on mobile to save

					vertical space.
QR Attendance Scanner	Receptionist	Fullscreen Viewport	Expo Camera (Mobile), HTML5 QR Reader (Web), Toast Notifications	Success: Green flash and Haptic feedback. Error: Red flash, buzzer sound.	Camera view fills the entire viewport; overlay controls sit at the bottom safe area.
Cashier Point of Sale (POS)	Cashier	2-Column Split (Search/Cart left, Checkout right)	User Search Combobox, Fee Breakdown DataTable, Print Dialog	Empty: "Search student to begin" graphic. Success: Thermal print trigger.	Search bar becomes persistently sticky at the top on smaller devices.
Course Detail & Syllabus	Student	Hero Header + Tabbed Interface	Shadow Tabs, Accordion (Syllabus tree), Floating Action Button	Loading: Text skeletons. Error: Error boundary with "Retry" button.	Hero image height is reduced by 50% on mobile; tabs become a scrollable horizontal list.
Secure Video Player	Student	Fullscreen Overlay	Custom Video Controls, DRM Watermark Overlay, Next Up	Loading: Buffering spinner. Error: "Playback failed"	Auto-rotates to landscape mode when fullscreen is triggered on mobile

			Card	overlay.	devices.
Bank Slip Verification Queue	Finance	Dense Data Grid	Shadow Table, Pagination, Image Preview Modal, Status Badge	Empty: "No pending slips" illustration.	Horizontal scroll enforced on mobile to preserve table column integrity.
Z-Score Analytics	Student/Parent	Dashboard Panel	Recharts (Scatter Plot), Input Forms (μ , σ), Metric Cards	Loading: Chart skeleton. Error: "Calculation failed" message.	Scatter plot charts transform into swipeable summary cards on mobile screens.

4. Authentication Module

The authentication module is the most heavily scrutinized gateway of the application, requiring exceptional frontend resilience. Built using React Hook Form and validated strictly via Zod schemas, the forms ensure that no malformed data reaches the backend. Registration for the Sri Lankan context requires strict mobile number validation utilizing standard regional regex patterns (Regex: `/(?:0|94|\+94)?(?:7\d{8})$/`) to ensure SMS delivery compatibility¹.

The OTP verification interface relies on a six-digit split-input component that automatically advances focus as the user types, accompanied by a 60-second cryptographic countdown timer that disables the resend button to prevent SMS gateway abuse. A critical frontend requirement is the Device Limit Warning system; because account sharing is rampant in the local tuition industry, the backend enforces active session limits. If a student attempts to log in from a third device when only two are permitted, the frontend intercepts the standard login resolution and mounts a modal overlay displaying the currently active devices, their IP locations, and last active timestamps. The user must use this interface to remotely terminate an existing session before the current login can proceed. Furthermore, the Session Expired flow must utilize a non-dismissible modal overlay that captures the current route state, pauses any active background tasks (such as video playback or exam timers), and requires inline re-authentication, completely avoiding a jarring redirect to the root login page.

5. Dashboard Design

Dashboards across the platform are engineered as modular assemblies of specialized widgets, fetching data asynchronously and independently via React Query to ensure rapid perceived load times. The Student Dashboard prioritizes active engagement, featuring a "Continue Learning" card that prominently displays the last accessed video or document with a localized progress bar and a one-click resume action. Below this, an upcoming schedule timeline visualizes live Zoom classes and physical batch times, ensuring students are constantly aware of immediate obligations. A unique feature for the A/L demographic is the Target Z-Score Tracker, which contextualizes the student's current MCQ average against the target standard deviations required for competitive university streams, utilizing Recharts to map their trajectory².

Conversely, the Institute Owner Dashboard is a data-dense command center. It features an interactive revenue area chart that smoothly transitions between daily, weekly, and monthly fee collections using Framer Motion for SVG path morphing. An attendance heatmap visualizes student presence across the calendar year, utilizing color intensity to highlight peak absence days, empowering owners to identify drop-out risks before they impact revenue. Quick action bars remain fixed at the top of the viewport, providing instant access to critical operations regardless of scroll position.

6. Course Management

The course management frontend allows administrative staff and teachers to digitally mirror the complex, multi-tiered structure of the Sri Lankan A/L curriculum. The user interface utilizes a nested Draggable Tree component, built upon modern drag-and-drop libraries, to allow visual organization of Courses into Streams (e.g., Physical Science, Biological Science, Commerce, Arts), Subjects, and specific chronological Batches.

Creating or editing a course triggers a comprehensive multi-step wizard formatted as a Stepper UI. This wizard handles metadata input, rich text descriptions, and a sophisticated thumbnail upload interface that includes client-side image cropping to enforce aspect ratio consistency across the platform. The pricing configuration step handles complex fee structures, allowing institutes to toggle between full upfront payments and automated monthly installment plans. Furthermore, the Timetable Manager provides a drag-and-drop calendar interface, wrapping FullCalendar, which allows teachers to schedule both physical classroom sessions and online WebRTC links. The frontend calculates potential overlaps in real-time, instantly generating conflict-detection toast notifications if a teacher attempts to double-book a resource or time slot⁴.

7. Student Learning Experience

The student learning interface is meticulously designed to minimize cognitive overload and digital distractions, maximizing educational engagement. The primary layout features a collapsible left sidebar containing the syllabus tree, which tracks completion status via optimistic UI updates; when a student marks a lesson as complete, the checkbox and overall

progress ring update immediately, rolling back only if the server responds with an error. The central content area dynamically hosts the Video Player, PDF Viewer, or Quiz interfaces without full page reloads, maintaining a fluid Single Page Application (SPA) experience.

A collapsible right sidebar houses interactive tools, including the AI chat interface, community discussion threads, and a personal markdown-supported notes editor. For the mobile application, a dedicated "Downloads" tab serves as the hub for offline learning. This interface displays encrypted videos and PDFs saved to the device's local storage, accompanied by a storage quota UI that visualizes the app's footprint on the device, allowing students to manage their offline assets efficiently.

8. Video Player

The video player represents the most critical intellectual property protection asset within the platform, demanding a highly secure, custom-engineered frontend wrapper around secure streaming technologies like VdoCipher⁵. The frontend discards native HTML5 controls in favor of a Tailwind-styled custom interface, ensuring cross-browser consistency and branding alignment.

To combat piracy, the web application implements a dynamic floating watermark component via a React DOM overlay. This watermark, displaying the authenticated student's unique ID and registered phone number, utilizes a randomized coordinate generator to move across the video viewport at ten-second intervals, effectively neutralizing attempts to use localized blurring software to obscure the identity⁶. On the mobile frontend built with React Native (Expo), the expo-screen-capture plugin is strictly enforced. The `preventScreenCaptureAsync()` method is invoked upon mounting the video component, leveraging the `FLAG_SECURE` window flag on Android and equivalent protections on iOS, forcing a black screen if the operating system attempts to record the screen or capture an app-switcher snapshot⁸. Player features include seamless quality selection (Auto, 1080p down to 360p to accommodate varying Sri Lankan bandwidths), playback speed controls (0.5x to 2.0x), Picture-in-Picture (PiP) support, and an interactive bookmarking system that allows students to drop timestamped notes directly onto the video progress bar.

9. PDF Viewer

The PDF Viewer is optimized for rendering A/L past papers, extensive model papers, and detailed tutes. To deter casual copying and unauthorized distribution, the frontend utilizes the react-pdf library to render document pages strictly as HTML5 canvas elements rather than selectable text layers. Standard browser context menus (right-click) and keyboard shortcuts for copying are programmatically disabled within the viewer's bounding box.

The user interface features a collapsible thumbnail sidebar for rapid multi-page navigation. Zoom controls allow magnification up to 400% without quality degradation, crucial for intricate mathematical and scientific diagrams. A custom text highlight tool saves relative coordinate data to the backend, allowing annotations to persist across sessions. Additionally, a specialized Dark Mode toggle applies specific CSS filter properties (`invert(1) hue-rotate(180deg)`) to the canvas elements, inverting the stark white backgrounds to dark gray while mathematically

preserving the original hues of embedded images and graphs, significantly reducing eye strain during late-night study sessions.

10. Payment Module

The payment interface is heavily contextualized to address Sri Lankan financial behaviors, seamlessly blending digital gateway integrations with robust manual deposit workflows. The online checkout flow utilizes a sliding drawer UI that presents a transparent breakdown of fees, separating tuition costs from admission or resource fees.

Given the high prevalence of physical bank deposits in the region, the frontend provides a dedicated Bank Slip Upload mechanism. This interface features a responsive drag-and-drop zone¹ and integrates a client-side image compression utility (e.g., browser-image-compression) to ensure uploads from high-resolution mobile cameras remain under a 1MB payload limit, saving backend storage and bandwidth. For in-person transactions, the Cashier Point of Sale (POS) interface is entirely keyboard-optimized, allowing receptionists to rapidly search student IDs via a combobox, process cash payments, and trigger local thermal receipt printing¹⁰. The status of all transactions is visualized throughout the platform using color-coded Shadcn badges: Pending (Yellow), Verified (Green), Failed (Red), and Overdue (Gray).

11. Attendance

The attendance module is engineered for extreme high-throughput scenarios, specifically designed to handle the rapid influx of hundreds of students entering a tuition hall within a narrow time window. The administrative web view includes a robust ID Card generation tool, dynamically rendering print-ready PVC card templates (standard CR80 size) that embed high-density, cryptographically signed QR codes unique to each student¹.

The mobile scanning interface, deployed on devices operated by receptionists, leverages the expo-camera module with optimized frame processors to ensure continuous, zero-lag barcode and QR scanning. Upon a successful scan, the UI immediately provides multi-sensory feedback to the operator: a bright green visual flash across the screen, a short vibration utilizing the device's Haptic Feedback API, and an optimistic increment of the on-screen counter¹⁰.

Concurrently, the web-based Live Analytics dashboard provides the Branch Manager with real-time entry statistics, visualizing hall capacity limits and peak entry velocity.

12. Examination Module

The examination interface is tailored to replicate the intense, time-sensitive nature of the G.C.E. A/L testing environment. The Multiple Choice Question (MCQ) interface employs a responsive split-view layout. On desktop, the left pane displays the PDF question paper, while the right pane houses a digital Optical Mark Recognition (OMR) grid for rapid answer selection. A highly visible, sticky countdown timer is anchored to the top of the viewport, turning red during the final five minutes to induce appropriate urgency.

Following the publication of results, the module unlocks the Z-Score Analytics Dashboard. Because A/L university admission in Sri Lanka is heavily reliant on statistical standardization

rather than raw marks, this frontend feature is critical. The interface receives the student's raw marks alongside the state-provided mean (μ) and standard deviation (σ) injected into the application context. It calculates the Z-Score on the client side using the standard formula $Z = \frac{X - \mu}{\sigma}$ and utilizes Recharts to visually plot the student's exact position on a bell curve relative to their peers². A gamified leaderboard table further contextualizes performance by displaying top rankings segmented by district and island-wide metrics.

13. Parent Portal

The Parent Portal is designed as a frictionless, heavily read-optimized interface catering to users who may have lower digital literacy. The layout utilizes large typography and highly legible data visualizations to display their child's attendance logs, upcoming fee deadlines, and comparative exam performance charts.

A comprehensive communication preference center allows parents to explicitly toggle their desired notification channels, selecting between SMS and email alerts for specific triggers such as unexpected absences or imminent fee reminders¹⁰. The interface avoids complex navigation trees, relying instead on a flat tab structure that surfaces all critical information within one or two clicks from the dashboard.

14. AI Features

The platform integrates advanced AI features surfaced through modern, highly responsive frontend interfaces. The AI Chatbot, designed for automated doubt clearance, utilizes a floating chat widget modeled after contemporary Large Language Model (LLM) interfaces. To handle latency, the frontend parses Server-Sent Events (SSE) to stream text responses character-by-character, creating a conversational feel. Crucially for A/L science and mathematics students, the chat interface integrates react-katex to seamlessly render complex LaTeX mathematical formulas and physics equations generated by the AI.

A Smart Search omnibar, accessible globally via the Cmd+K or Ctrl+K keyboard shortcut, leverages powerful frontend search clients (such as Algolia or Meilisearch) to provide instant, as-you-type results across video transcripts, PDF text content, and course syllabi. Additionally, a Flashcard module generates swipeable card components, utilizing fluid Tinder-style gesture interactions on mobile devices, allowing students to rapidly review AI-generated summaries of their lecture notes.

15. Notifications

The notification architecture ensures that critical alerts reach users immediately across all devices. Push notifications are handled via Expo Notifications for the mobile application and integrated Service Workers for Web Push on supported desktop browsers.

The web interface features an Announcement Center, accessed via a dropdown bell icon in the global header. Notifications within this popover are chronologically grouped (Today, Yesterday, Last Week) and categorized by type (System Alerts, Course Updates, Payment Reminders). Unread states are clearly indicated by increased font weight and a highly visible

primary-colored dot indicator. A toast notification system, utilizing a customized sonner or react-hot-toast implementation, handles ephemeral success and error messages throughout the application without requiring user dismissal.

16. Mobile App (React Native/Expo)

The mobile application, built entirely with React Native and the Expo framework, is optimized primarily for Android devices, which dominate the Sri Lankan demographic. The navigation paradigm utilizes @react-navigation/bottom-tabs for immediate access to primary functions (Home, Learn, Downloads, Profile) and a hidden drawer for auxiliary settings and administrative links.

A critical requirement is robust Offline Mode capability. The app leverages expo-file-system to securely store encrypted video chunks and PDF documents locally on the device. State persistence is managed synchronously via Zustand integrated with react-native-mmkv for high-performance key-value storage. The application actively monitors network connectivity using @react-native-community/netinfo. Upon detecting a connection drop, the UI seamlessly transitions into Offline Mode, displaying a prominent banner, disabling online-only tabs, and routing the user directly to their localized Download Manager where they can consume previously saved content without interruption.

17. Design System

The platform's visual language utilizes a customized, Material Design-inspired aesthetic layered precisely over Tailwind CSS utility classes, ensuring strict adherence to WCAG 2.1 AA accessibility standards for color contrast and screen reader compatibility.

The color palette is meticulously defined in the Tailwind configuration file:

- **Primary Brand:** Indigo (#4F46E5), used for primary actions, active states, and primary brand moments.
- **Secondary/Accent:** Teal (#0D9488), utilized for success states and secondary highlights.
- **Destructive:** Rose (#E11D48), strictly reserved for error states, destructive actions (e.g., deleting a course), and failed payments.
- **Backgrounds:** Slate 50 (#F8FAFC) for light mode interfaces, and Slate 950 (#020617) for the comprehensive dark mode implementation.

Typography relies on the highly legible Inter font family for all user interface elements, buttons, and navigation, while switching to the serif Merriweather font for long-form reading contexts within the PDF viewer to reduce visual fatigue. Spacing and auto-layout rules strictly adhere to a consistent 8px grid system, ensuring vertical and horizontal rhythm across all viewports.

18. Component Library

The frontend architecture relies on a proprietary component library built upon the foundation of Shadcn UI and Radix Primitives, ensuring accessibility and unstyled flexibility. Every component is strongly typed with TypeScript interfaces defining acceptable props, variants, and states.

Component Name	Purpose	Key Props & Variants	State Behaviors	Accessibility & Responsiveness
DataTable	Global data display (Users, Payments, Courses)	columns, data, searchable, pagination. Variants: compact, spacious.	Manages internal sorting and filtering state before passing to React Query.	Uses ARIA grid roles. Converts to stacked cards on viewports under 640px.
ResponsiveDialog	Contextual modals and forms	isOpen, onClose, title, description.	Traps focus within the modal when open. Handles escape key dismissal.	Renders as a centered Modal on desktop; morphs into a bottom-anchored vault drawer on mobile.
VideoCard	Displaying course assets in grids	thumbnailUrl, duration, title, progress. Variants: grid, list.	Displays a skeleton loader if thumbnailUrl is null. Shows hover play icon.	aria-label includes title and duration. Image scales proportionally.
StatCard	Dashboard metric visualization	icon, title, value, trendPercentage.	Animates the value counting up from zero on initial mount.	Screen readers announce the trend as "increased by X percent".
OTPInput	Secure authentication	length (default 6), onComplete.	Auto-advances focus to the next input cell on keydown.	Supports paste events, automatically distributing characters

				across cells.
--	--	--	--	---------------

19. State Management

The frontend state architecture is strictly segregated to prevent synchronization bugs and ensure high performance.

Server State (React Query): All asynchronous data fetching, caching, background polling, and cache invalidation are handled by TanStack Query.

- Query Keys follow a rigid, predictable taxonomy to allow precise invalidation: ['entity', 'id', 'action'] (e.g., ['student', 123, 'attendance']).
- Mutations automatically invalidate relevant query keys on success, triggering background refetches without requiring hard page reloads.

Client State (Zustand): Synchronous UI state and globally accessible preferences are managed via lightweight Zustand stores.

- useAuthStore: Persists the JWT, decodes user roles, and manages the boolean flags for device limit warnings.
- useVideoPlayerStore: Manages the current video timestamp, playing state, and global playback speed. This store allows the video player to detach into a Picture-in-Picture mini-player that persists as the user navigates across different React Router routes.
- useThemeStore: Toggles between light, dark, and system default preferences, directly manipulating the dark class on the root HTML element.

20. Responsive Behaviour

The application is engineered using a mobile-first philosophy, utilizing Tailwind's minimum-width breakpoints to progressively enhance the layout for larger displays.

- **Mobile Viewports (< 640px):** The UI enforces a single-column layout. Main navigation is collapsed into a hamburger menu utilizing a sliding sheet component. Complex data tables are hidden, replaced by stacked card representations of the data to prevent horizontal scrolling breakage.
- **Tablet Viewports (640px - 1024px):** Layouts transition to 2-column grids. Sidebars become collapsible icon-only rails to maximize horizontal screen real estate for content consumption.
- **Desktop Viewports (> 1024px):** Full complex data grids are rendered. Sidebars become persistent and fully expanded. Dashboards utilize 3 or 4-column masonry layouts.
- **Large Displays (> 1536px):** Maximum width constraints (max-w-screen-2xl) are applied to the main content blocks to ensure UI elements, text line lengths, and video players do not stretch uncomfortably across ultra-wide monitors, keeping the interface centered and legible.

21. UX Details

The frontend specification mandates a highly polished user experience achieved through

meticulous attention to micro-interactions and perceived performance.

Optimistic UI patterns are heavily enforced. When a user performs an action such as "Liking a comment" or "Marking attendance," the React component immediately updates the visual state before the API request resolves. If the network request subsequently fails, a toast notification alerts the user, and the UI state gracefully rolls back to its previous iteration. Button components feature subtle scale-down transforms utilizing CSS transitions (`active:scale-95`) to provide immediate physical feedback upon clicking.

Keyboard shortcuts are mapped globally using `react-hotkeys-hook`, allowing power users to navigate efficiently. For example, pressing Space toggles video play/pause, F engages fullscreen mode, and `Cmd+K` summons the global search palette. To optimize perceived loading times, the application strictly avoids generic spinning loaders for full-page data fetches. Instead, customized Skeleton loaders, built to match the exact dimensions of the anticipated text and image blocks, are rendered immediately, preventing jarring layout shifts when the data finally resolves.

22. Complete User Flows

To ensure developers understand the holistic experience, the frontend must support uninterrupted user journeys across multiple modules.

Flow: Student Course Acquisition and DRM Playback The student lands on the public Institute Portal and navigates to the Course Catalog. Upon selecting a specialized subject (e.g., Combined Mathematics), they view the course syllabus and pricing. Clicking "Enroll" triggers the Sliding Checkout Drawer. The student selects "Bank Deposit" as the payment method, dragging and dropping a photograph of their physical bank slip into the designated dropzone¹. The UI compresses the image, uploads it, and transitions the course state to "Pending Verification." Once the Branch Manager approves the slip via the Admin Dashboard, the student receives an instant push notification. Clicking the notification opens the mobile app directly to the Course Detail view. The student clicks the first video lesson. The frontend verifies the `useAuthStore` session and mounts the video player. Simultaneously, the React Native application executes `preventScreenCaptureAsync()` to secure the screen⁸. If the student attempts a screenshot, the OS captures only a black rectangle, and the frontend logs a warning toast to the screen.

23. Figma Structure

Alignment between the design and development teams requires a strictly organized Figma workspace that perfectly mirrors the React component architecture. The Figma file must be segmented into distinct Pages: `_Cover`, `_Design System` (containing all core tokens and typography), `Admin Web`, `Student Web`, `Mobile App (iOS/Android)`, and `_Archive`.

Component naming conventions in Figma must map directly to the frontend repository structure using the format: `[Category] / [Component] / [Variant] - [State]` (e.g., `Inputs / TextField / Outline - Error`). All UI screens and components must utilize Figma's Auto Layout feature extensively, simulating CSS Flexbox behavior precisely as the Tailwind utility classes will render it in the browser, ensuring exact padding and margin replication.

24. Deliverables

The execution of this specification will result in a production-ready frontend architecture. To successfully close out the development cycle, the frontend must pass rigorous UI Acceptance Criteria. Lighthouse performance scores must consistently exceed 90 on Desktop for authenticated routes. The application must pass automated WCAG 2.1 AA accessibility contrast checks. All forms must demonstrably prevent submission and display accurate inline error messages if Zod validation schemas are violated. Furthermore, mobile screens must be completely devoid of unintended horizontal scrolling.

Development rollout will follow a prioritized timeline. **Phase 1 (MVP)** will deliver the core Authentication flows, basic Role Dashboards, the Course Catalog, the Secure Video Player (incorporating core DRM mechanisms), and the Payment Checkout system featuring the Bank Slip upload functionality. **Phase 2 (V1)** will introduce the high-throughput QR Attendance scanner, the Examination MCQ module, and the custom PDF Viewer. **Phase 3 (V2)** will deploy the advanced Z-Score analytics engines, the AI Chatbot integrations, the Parent Portal, and the complex local file system integrations required for Mobile Offline Downloads.

Works cited

1. tuitionfee.lk - Sri Lanka's #1 Most Affordable Tuition Class Payment & Attendance Management System, <https://tuitionfee.lk/>
2. The “Z-Score” and the fate of A/L students - News Features | Daily Mirror, <https://www.dailymirror.lk/news-features/The-Z-Score-and-the-fate-of-AL-students/131-201523>
3. Sri Lanka A/L Z-Score Calculator (2024 μ/σ) | induwara.lk, <https://induwara.lk/tools/sri-lanka-al-z-score-calculator>
4. Tuition Class Management System Sri Lanka | Class Scheduling, <https://possystem.lk/tuition-class-management-system>
5. VdoCipher: Secure Video Hosting, DRM & Custom Player, <https://www.vdocipher.com/>
6. Create a floating video player in React - javascript - Stack Overflow, <https://stackoverflow.com/questions/74477049/create-a-floating-video-player-in-react>
7. How to Add Dynamic Watermark to the Video Player | DynTube Documentation, <https://www.dyntube.com/docs/tips-and-tricks/how-to-add-dynamic-watermark-to-the-video-player>
8. Expo ScreenCapture - Expo Documentation, <https://docs.expo.dev/versions/latest/sdk/screen-capture/>
9. Testing Screenshot Prevention in React Native: Simulators, Real Devices, and Edge Cases, <https://prateeksha.com/blog/testing-screenshot-prevention-react-native>
10. Features - Accept class Payments and mark Tuition Class Student Attendance with a ඊයේ swipe of a card., <https://cfms.lk/features.html>
11. Best ERP Software - School Management System Sri Lanka, <https://possystem.lk/blogs/school-management-system-sri-lanka>